

桌面宠物 — 设计文档

2026 年 6 月 6 日

摘要

本文档介绍了一个基于 Windows GDI+ 与 WebView2 的桌面宠物程序的设计与实现。该程序使用分层窗口（Layered Window）技术渲染带 Alpha 通道的 PNG 图片，在屏幕上显示一只可交互的宠物，支持拖拽、点击反馈、物理下落、地面漫游以及 AI 聊天等功能。

目录

1	项目概述	2
1.1	功能特性	2
1.2	开发环境	2
2	系统架构	2
3	核心技术	3
3.1	分层窗口渲染	3
3.2	物理与状态机	4
3.3	WebView2 集成	4
3.4	窗口置顶优先级	5
4	图片资源管理	5
4.1	目录结构	5
4.2	权重随机机制	6
5	编译与运行	7
5.1	前置条件	7
5.2	编译命令	7
5.3	运行	7
6	核心数据结构	7
6.1	PetState — 宠物物理状态	7
6.2	GroundTask — 地面行为枚举	7
6.3	ClickImageItem — 可配置的图片资源	8
7	改进与优化	8

1 项目概述

1.1 功能特性

- 拟物物理：宠物受简单重力影响可下落，碰到地面后在地面上左右漫游。
- 拖拽交互：用户可拖拽宠物，松开后根据拖拽力度和方向产生初速度。
- 点击反馈：单击宠物时按权重随机切换显示不同图片。
- 方向感知：行走、拖拽、下落等状态均区分左/右方向，自动选择对应文件夹 (photo_left/、photo_right/) 中的图片。
- AI 聊天：双击宠物打开聊天窗口，基于 WebView2 加载 HTML 聊天界面 [1]，通过 JavaScript 与 AI 后端通信。
- 可视化配置：右键弹出设置窗口，可实时调整各图片的随机权重以及聊天颜色。
- 系统托盘：支持最小化到托盘，双击托盘图标切换显示/隐藏。

1.2 开发环境

- 编译器：MinGW-w64 (GCC) 8.1.0，使用 g++ 编译
- 图形库：GDI+ (Windows GDI Plus)
- Web 引擎：Microsoft WebView2 (Edge Chromium)
- 构建方式：命令行编译 / VS Code 任务
- 源码行数：约 2,400 行

2 系统架构

程序整体采用单一 C 源文件（实际使用 C++ 编译），以 WinMain 为入口，围绕 Windows 消息循环构建事件驱动架构。模块划分如下：

```

WinMain
    |-- GDI+ 初始化
    |-- 图片资源加载
    |-- 窗口创建 (WS_EX_LAYERED | WS_EX_TOPMOST)
    |-- 消息循环
        |-- WndProc (主窗口)
            |-- WM_TIMER -> StepPet + RenderPet
            |-- WM_LBUTTONDOWN/UP/MOVE -> 点击/拖拽
            |-- WM_LBUTTONDOWNBLCLK -> 聊天窗口
            |-- WM_RBUTTONUP -> 设置窗口
            |-- WM_WINDOWPOSCHANGING -> 强制置顶
            +-- WM_TRAYICON -> 托盘操作
        |-- ChatWndProc (聊天窗口)
            |-- WM_CREATE -> 创建子控件
            |-- WM_COMMAND -> 发送/配置按钮
            +-- WM_CTLCOLOR -> 自定义颜色
        |-- WebViewHostWndProc (WebView 容器)
            |-- WM_SIZE -> 调整 WebView 尺寸
            +-- WM_CLOSE -> 隐藏
        +-- SettingsWndProc (设置窗口)
            +-- WM_COMMAND -> 应用/关闭/颜色选择

```

3 核心技术

3.1 分层窗口渲染

宠物窗口使用 `WS_EX_LAYERED` 扩展样式创建，通过 `UpdateLayeredWindow` API 将包含 Alpha 通道的 32 位 DIB (Device Independent Bitmap) 直接绘制到屏幕上。配合 `WS_EX_TOPMOST` 实现置顶显示。

渲染流程如代码 1 所示：创建内存 DC → 创建 DIBSection → 使用 GDI+ Graphics 绘制 PNG → 调用 `UpdateLayeredWindow`。

```

1 static void RenderPet(HWND hwnd) {
2     // 1. 创建屏幕兼容 DC 和 32-bit DIBSection
3     HDC screenDC = GetDC(NULL);
4     HDC memDC = CreateCompatibleDC(screenDC);
5     BITMAPINFO bmi = {};
6     bmi.bmiHeader.biWidth = g_pet.width;

```

```

7     bmi.bmiHeader.biHeight = -g_pet.height; // 自上而下
8     bmi.bmiHeader.biPlanes = 1;
9     bmi.bmiHeader.biBitCount = 32;
10    bmi.bmiHeader.biCompression = BI_RGB;
11    HBITMAP dib = CreateDIBSection(screenDC, &bmi, DIB_RGB_COLORS, &
        bits,...);
12
13    // 2. 用 GDI+ 绘制 PNG (支持透明通道)
14    Graphics graphics(memDC);
15    graphics.SetCompositingMode(CompositingModeSourceCopy);
16    graphics.Clear(Color(0,0,0,0)); // 全透明底色
17    graphics.SetCompositingMode(CompositingModeSourceOver);
18    graphics.DrawImage(g_image, 0, 0, w, h); // 叠放 PNG
19
20    // 3. 更新分层窗口
21    BLENDFUNCTION bf = { AC_SRC_OVER, 0, 255, AC_SRC_ALPHA };
22    UpdateLayeredWindow(hwnd, screenDC, &pos, &size, memDC, &src, 0,
        &bf, ULW_ALPHA);
23 }

```

Listing 1: RenderPet 核心渲染流程

3.2 物理与状态机

宠物状态分为 **飞行** (airborne) 和 **地面** (ground) 两种。每帧 (WM_TIMER, 10ms) 执行 StepPet, 流程如下:

1. 更新位置: $x \leftarrow x + v_x$, $y \leftarrow y + v_y$
2. 边界检查: 碰到左右墙反弹 ($v_x \leftarrow -v_x$), 触地则 $v_y = 0$, 进入地面模式
3. 地面任务: 接地后随机选择一个地面任务 (定时左移/右移/到边缘/停止)
4. 图片切换: 根据方向 (v_x 符号) 和状态选择对应的 PNG

3.3 WebView2 集成

聊天功能通过 WebView2 加载本地 HTML 文件实现 [1]。通信采用双向 JSON 消息传递:

- C++ $\rightarrow$ JS: PostWebMessageAsJson 发送包含用户文本的 JSON
- JS $\rightarrow$ C++: 通过 webMessageReceived 事件接收 AI 回复

采用 COM 回调模式，定义了三个回调类：

- WebViewEnvCompletedHandler — 环境创建完成
- WebViewControllerCompletedHandler — 控制器创建完成
- WebViewMessageReceivedHandler — 接收 JS 消息

3.4 窗口置顶优先级

为解决宠物窗口被任务栏遮挡的问题，采用双层保障策略：

```

1 // 策略 1: 拦截 Z-order 变化
2 case WM_WINDOWPOSCHANGING: {
3     WINDOWPOS* wp = (WINDOWPOS*)lParam;
4     wp->hwndInsertAfter = HWND_TOPMOST;
5     wp->flags &= ~SWP_NOZORDER;
6     return 0;
7 }
8
9 // 策略 2: 定时器轮询 (每 500ms)
10 case WM_TIMER:
11     // ... 更新物理与渲染 ...
12     if ((GetTickCount() / 500) != g_lastTopmostCheck) {
13         g_lastTopmostCheck = GetTickCount() / 500;
14         SetWindowPos(hwnd, HWND_TOPMOST, 0, 0, 0, 0,
15                     SWP_NOMOVE | SWP_NOSIZE | SWP_NOACTIVATE);
16     }

```

Listing 2: 强制置顶实现

4 图片资源管理

4.1 目录结构

程序从 photo_left/ 和 photo_right/ 两个目录读取图片资源，基于当前移动方向自动选择：

```

Desktop pet/
|-- text1.c          # 源码
|-- output/text1.exe # 编译输出
|-- photo_left/      # 向左移动时使用的图片
|   |-- 0.png        # 行走

```

```

|      |-- 1.png      # 停止触发（权重可配）
|      |-- 2.png      # 低高度下落 / 落地
|      |-- 3.png      # 拖拽
|      |-- 4.png      # 停止触发（权重可配）
|      |-- 5.png      # 下落
|      |-- 6.png      # 点击触发（权重可配）
|      |-- 7.png      # 点击触发（权重可配）
|      +-- 8.png      # 落地特殊（权重可配）
|-- photo_right/      # 向右移动时使用的图片
|      +--（同上）
|-- ai_chat/          # AI 聊天界面
|      +-- Midsummer's Bird.html
+-- external/         # WebView2 SDK

```

4.2 权重随机机制

程序通过加权随机算法实现多样化反馈。以点击事件为例：

$$\text{选中图片 } i \text{ 的概率} = \frac{w_i}{\sum_j w_j} \quad (1)$$

其中 w_i 为第 i 张图片的权重，所有有效图片的权重和作为分母。各权重可在设置窗口中实时调整。代码 3 展示了实现：

```

1 static Image* PickClickImageByDirection(BOOL toLeft) {
2     int total = 0;
3     for (int i = 0; i < kClickItemCount; ++i) {
4         Image* img = toLeft ? g_clickItems[i].left : g_clickItems[i].
5             right;
6         if (img && g_clickItems[i].weight > 0)
7             total += g_clickItems[i].weight;
8     }
9     if (total <= 0) return NULL;
10    int r = rand() % total;
11    for (int i = 0; i < kClickItemCount; ++i) {
12        Image* img = toLeft ? g_clickItems[i].left : g_clickItems[i].
13            right;
14        if (img && g_clickItems[i].weight > 0) {
15            if (r < g_clickItems[i].weight) return img;
16            r -= g_clickItems[i].weight;
17        }
18    }
19    return NULL;

```

18 }

Listing 3: 加权随机选择

5 编译与运行

5.1 前置条件

- MinGW-w64 (g++) 或任何 C++17 编译器
- WebView2 Runtime (Windows 11 自带, Windows 10 需安装)
- GDI+ SDK (MinGW 自带)

5.2 编译命令

```
g++ -I "external/webview2/pkg/build/native/include" ^  
-g3 -mwindows text1.c -o output/text1.exe ^  
-lgdi32 -luser32 -lmsimg32 -lgdiplus -lole32 -luuid -lshell32
```

5.3 运行

双击 output/text1.exe 或从命令行启动。

6 核心数据结构

6.1 PetState — 宠物物理状态

```
1 typedef struct {  
2     int x, y;           // 屏幕坐标 (像素)  
3     int vx, vy;        // 速度 (像素/帧, 每帧约 16ms)  
4     int width, height; // 窗口尺寸  
5 } PetState;
```

6.2 GroundTask — 地面行为枚举

```
1 typedef enum {  
2     TASK_NONE,  
3     TASK_MOVE_LEFT_EDGE, // 向左走到边缘  
4     TASK_MOVE_RIGHT_EDGE, // 向右走到边缘
```

```

5     TASK_MOVE_LEFT_TIME,    // 向左走固定时长 (5s)
6     TASK_MOVE_RIGHT_TIME,   // 向右走固定时长 (5s)
7     TASK_STOP_TIME          // 停止 (1.5s), 仅在上一个任务为
8                             // MOVE_LEFT_TIME/RIGHT_TIME 后可触发
9 } GroundTask;

```

6.3 ClickImageItem — 可配置的图片资源

```

1 typedef struct {
2     const wchar_t* fileName; // 图片文件名
3     int weight;              // 随机权重
4     Image* left;             // 左方向时使用的 GDI+ Image
5     Image* right;            // 右方向时使用的 GDI+ Image
6 } ClickImageItem;

```

7 改进与优化

本代码经过以下优化处理：

1. **清理重复清理代码：**WinMain 末尾的图片资源释放原先重复 38 行，通过 `SAFE_DELETE_IMAGE` 宏压缩为 14 行。
2. **消除编译警告：**将结构体初始化 `{ 0 }` 统一改为 `{}`，消除所有“-Wmissing-field-initializers”警告；移除未使用的变量 `g_trayHwnd`。
3. **强制置顶：**添加 `WM_WINDOWPOSCHANGING` 拦截 + 定时器轮询，双重保障宠物不被任务栏遮挡。
4. **使用 C++ 编译器：**原代码使用 GDI+ 和 C++ 特性（`class`、`new/delete`、`reinterpret_cast`），必须用 `g++` 而非 `gcc` 编译。

参考文献

- [1] Soaring Bird. *Midsummer's Bird — An Elegant, Powerful, and Fully Localized AI Role-Playing Chat Frontend*. <https://github.com/SoaringBird/Midsummer-s-Bird>, 2025. MIT License.
- [2] SillyTavern. *SillyTavern — LLM Frontend for Power Users*. <https://github.com/SillyTavern/SillyTavern>.

- [3] Microsoft. *WebView2 — Microsoft Edge WebView2 Control*. <https://learn.microsoft.com/en-us/microsoft-edge/webview2/>, 2025.
- [4] MinGW-w64. *MinGW-w64 — GCC for Windows 64 & 32-bit*. <https://www.mingw-w64.org/>.